



Tecnológico de Monterrey

Campus Monterrey

Requerimientos Tecnicos

Aldo Karim García Zapata | A00843726 | Ingeniería en Tecnologías Computacionales

Profesor: Luis Salomon Flores Ugalde

24/09/2026

What is the problem and why is it relevant?

The problem consists of managing and analyzing a large log file that contains thousands of access attempts to a server, detailing the date, time, IP address, and the result (e.g., “Failed password” or “Illegal user”).

In a real-world scenario, system administrators and cybersecurity teams must quickly detect anomalies, audit accesses, and find specific intrusion attempts within massive amounts of data.

Processing these logs manually is virtually impossible due to the sheer volume of information (over 16,000 records). Therefore, a programmatic solution capable of organizing and filtering these records efficiently by date range is highly relevant and crucial for timely incident response.

How was the problem solved?

The problem was solved by building a C++ application that performs the following steps:

Data Loading: The application reads bitacora.txt sequentially and stores each line in primary memory. To facilitate mathematical comparisons, the date strings are translated into numerical format.

Sorting: The entire dataset is sorted chronologically in ascending order using the numeric representation of the dates.

User Query: The system prompts the user to input a starting date and an ending date.

Fast Retrieval: Using a binary search approach, the program finds the indices of the first and last records that fall within the specified date boundaries.

Data Export: The records found within the requested range are collected and written to an output file

What data structures and algorithms were used

Data Structures:

- `std::vector<string>` : Used to store the complete raw text of each line.
- `std::vector<int>` : A vector used to store the numeric representation of the dates acting as sorting keys.

Algorithms:

- Merge Sort: Used to order the dates vector chronologically.
- Binary Search: Used to quickly find the lower and upper bounds of the requested date range within the already sorted data.

How were they implemented and why were they selected?

Merge Sort Implementation: It was implemented recursively, dividing the arrays into two halves until they reach a size of 1, and then merging them back together in an ordered sequence

Why selected? Merge sort is highly predictable and provides a guaranteed time complexity regardless of the initial state of the data. Since log files can be partially ordered or entirely chaotic, ensuring a consistent sorting time is ideal.

Binary Search Implementation: It was implemented iteratively, constantly halving the search space until the target key is located.

Why selected? Linearly searching through 16,000+ records every time the user makes a query would be slow. Binary search leverages the fact that the data is already sorted, dramatically cutting down the number of comparisons needed to find the exact boundaries.

What are their advantages and disadvantages

Merge Sort

- **Advantages:** It is highly efficient and stable. It handles large datasets gracefully and does not suffer from worst-case degradation like Quick Sort.
- **Disadvantages:** It has a significant memory overhead. The merge step requires creating temporary sub-arrays, doubling the memory footprint during execution (Space complexity of $O(n)$).

Binary Search

- **Advantages:** Exceptional speed for lookups. It handles enormous datasets with minimal steps.
- **Disadvantages:** It strictly requires the data to be pre-sorted. If a new log entry is appended, the dataset must be re-sorted before binary search can be reliably used again.

What is their computational complexity in the best and worst cases, and what determines it?

Merge Sort

- **Best, Average, and Worst Case Time Complexity:** $O(n \log n)$
- **What determines it?** The algorithm always divides the array of size n into halves. The depth of this division tree is always $\log_2(n)$. At each level of the tree, merging the elements takes linear time $O(n)$. Therefore, the complexity is constantly $n \log(n)$.

Binary Search

- Best Case Time Complexity: $O(1)$ (occurs if the middle element of the array happens to be the exact target value on the first try).
- Worst Case Time Complexity: $O(\log n)$ (occurs when the target is at the very edges of the array or not present at all).
- What determines it? The search space is divided by 2 in each iteration. For an array of size N , the maximum number of divisions before the space is reduced to 1 is $\log_2(N)$.

What possible improvements could be implemented?

Year Handling: The current timestamp extraction logic MMDDHHMMSS completely ignores the year. If logs cross over a New Year, the sorting would fail structurally.

Avoiding RAM limitations: Reading a text file into RAM is acceptable for 16,000 lines, but for industrial log files spanning Gigabytes, it would crash. An improvement would be to parse the file line by line without storing the text strings, maintaining an external index of file byte-offsets instead.

References consulted

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

C++ Reference. (n.d.). `std::vector`. Retrieved from <https://en.cppreference.com/w/cpp/container/vector>

C++ Reference. (n.d.). `std::ifstream`. Retrieved from https://en.cppreference.com/w/cpp/io/basic_ifstream